

Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia de Telecomunicações e Informática

Serviços de Rede e Aplicações Multimédia

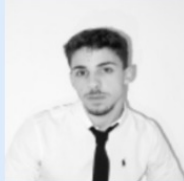
Trabalho Prático 2

**Streaming de Vídeo e Redes de Distribuição
de Conteúdo**

Trabalho realizado por:



pg59972
Bernardo Gualdino
Salgado
pg59972@alunos.uminho.pt



pg61434
Luís Pedro Cerqueira
Baptista
pg61434@alunos.uminho.pt

Guimarães (Azúrem), 31 de maio de 2026

Conteúdo

1	Introdução	3
2	Etapa 1: Streaming HTTP simples sem adaptação	3
2.1	Topologia e Conectividade	3
2.2	Implementação e Análise de Tráfego	4
2.3	Conclusão da Escalabilidade	5
3	Etapa 2: Streaming adaptativo sobre HTTP (MPEG-DASH)	6
3.1	Implementação e Configuração	6
3.2	Análise do Manifesto e Funcionamento (Questões 2 e 4)	7
3.3	Adaptação ao Congestionamento de Rede (Questão 3)	8
4	Etapa 3: Streaming RTP/RTCP em Unicast e Multicast	11
4.1	Configuração e Análise Unicast	11
4.2	Configuração e Análise Multicast	13
4.3	Comparação de Escalabilidade (Questão 5)	14
5	Etapa 4: Redes de Overlay e Distribuição de Conteúdo	15
5.1	Implementação da Rede Base (Underlay)	15
5.2	Implementação da Rede Overlay	16
5.3	Análise de Desempenho	17
5.4	Conclusões Comparativas	18
6	Conclusões	19
7	Referências	19
8	Declaração de Uso de IA e Responsabilidade	20

Lista de Figuras

1	Topologia da rede emulada no CORE.	3
2	Teste de Ping.	4
3	Teste de Traceroute.	4
4	Carga de CPU no servidor durante a transmissão (2,0%).	5
5	Captura Wireshark - Cenário com 1 Cliente.	6
6	Captura Wireshark - Cenário com 2 Clientes.	6
7	Captura Wireshark - Cenário com 3 Clientes.	6
8	Captura Wireshark: Pedido inicial do ficheiro manifesto e bibliotecas JavaScript.	7
9	Configuração do limite de largura de banda para 300 kbps.	8
10	Topologia CORE com indicação do limite aplicado.	8
11	Tráfego no nó Bela antes da limitação: pedidos dos segmentos de 1000 kbps.	9
12	Tráfego no nó Bela após a limitação: alteração progressiva para a versão de 200 kbps.	9
13	Tráfego no nó Aladdin: pedidos da versão de 1000 kbps.	10
14	QoE no Aladdin: Qualidade 640x400.	11
15	QoE no Bela: Qualidade 160x100.	11
16	Pacotes UDP enviados para o cliente (Unicast).	12
17	Tráfego interpretado como RTP.	12
18	Estatísticas UDP: Tráfego gerado para 1 cliente Unicast ($\approx$ 209 kbps).	12
19	Estatísticas UDP: Tráfego gerado para 2 clientes Unicast simultâneos ($\approx$ 424 kbps).	12
20	Topologia de rede utilizada no cenário Multicast.	13
21	Pacote de anúncio SAP/SDP e transmissão da <i>stream</i> de vídeo (MP4V-ES).	13
22	Captura de pacotes UDP no cenário Multicast.	14
23	Estatísticas UDP: Tráfego total do servidor no cenário Multicast com 4 clientes ($\approx$ 211 kbps).	14
24	Topologia base da rede com 5 <i>routers</i> e múltiplos nós.	15
25	Teste ping : Servidor para PC8.	16
26	Teste traceroute : Caminho pela rede.	16
27	Consumo de CPU no nó retransmissor provocado pelo processo FFmpeg.	17
28	Wireshark: Entrada do fragmento de vídeo no nó retransmissor (13.221914s).	18
29	Wireshark: Saída do fragmento reencaminhado para o cliente (13.221969s).	18

1 Introdução

Neste documento descrevem-se as atividades e os resultados do Trabalho Prático 2. O objetivo consistiu no estudo de sistemas de *streaming* de vídeo a pedido e em tempo real, com recurso ao emulador CORE [1]. Analisaram-se as opções da pilha protocolar e o desempenho da rede em diversos cenários.

Para o transporte dos fluxos de vídeo utilizaram-se ferramentas como o FFmpeg [3], para o processamento de ficheiros, e o VLC [4], como emissor e cliente. A recolha de dados de tráfego para análise foi feita com o Wireshark [2].

O relatório divide-se em quatro etapas. Na Etapa 1 analisou-se o *streaming* HTTP simples sem adaptação de débito. Na Etapa 2 estudou-se o *streaming* adaptativo sobre HTTP através da norma MPEG-DASH [5]. Na Etapa 3 comparou-se a eficiência do protocolo UDP nos modos *Unicast* e *Multicast*. Na Etapa 4 avaliou-se a implementação de uma rede *Overlay* para o encaminhamento de fluxos MPEG-TS sobre UDP.

2 Etapa 1: Streaming HTTP simples sem adaptação

2.1 Topologia e Conectividade

Nesta primeira fase do trabalho prático, foi construída a topologia base no emulador CORE, constituída por um servidor de vídeo (*VStreamer*), dois *switches* (sw1, sw2), dois *routers* OSPF (router1, router2) e quatro clientes finais (*Jasmine*, *Aladdin*, *Bela* e *Monstro*). A Figura 1 ilustra a rede configurada.

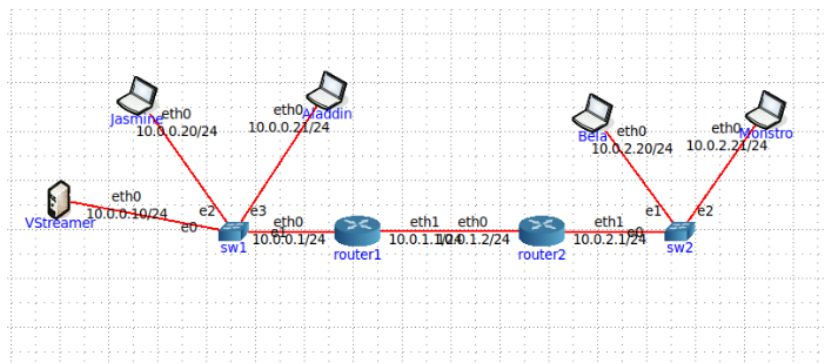
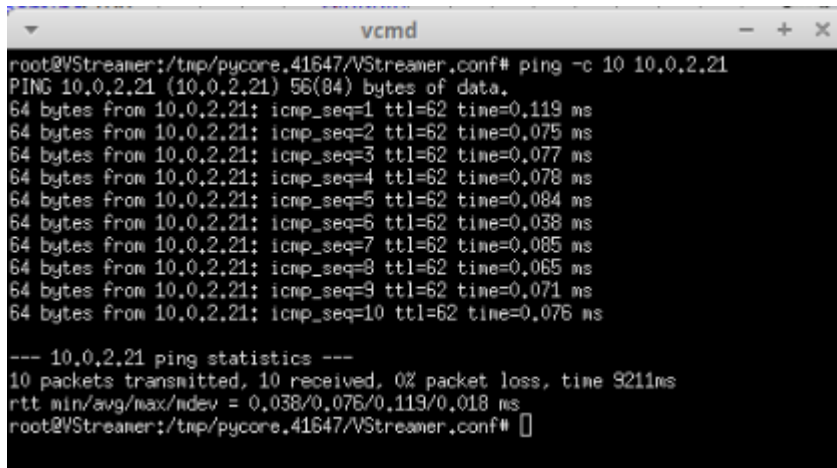


Figura 1: Topologia da rede emulada no CORE.

Para validar a correta configuração do encaminhamento entre as diferentes sub-redes, efetuaram-se testes de conectividade ICMP (*ping*) e *traceroute* entre o servidor (10.0.0.10) e o cliente *Monstro* (10.0.2.21), confirmando-se a comunicação fim-a-fim com sucesso, conforme visível nas Figuras 2 e 3.



```
vcmd
root@VStreamer:/tmp/pycore.41647/VStreamer.conf# ping -c 10 10.0.2.21
PING 10.0.2.21 (10.0.2.21) 56(84) bytes of data.
64 bytes from 10.0.2.21: icmp_seq=1 ttl=62 time=0.119 ms
64 bytes from 10.0.2.21: icmp_seq=2 ttl=62 time=0.075 ms
64 bytes from 10.0.2.21: icmp_seq=3 ttl=62 time=0.077 ms
64 bytes from 10.0.2.21: icmp_seq=4 ttl=62 time=0.078 ms
64 bytes from 10.0.2.21: icmp_seq=5 ttl=62 time=0.084 ms
64 bytes from 10.0.2.21: icmp_seq=6 ttl=62 time=0.038 ms
64 bytes from 10.0.2.21: icmp_seq=7 ttl=62 time=0.085 ms
64 bytes from 10.0.2.21: icmp_seq=8 ttl=62 time=0.065 ms
64 bytes from 10.0.2.21: icmp_seq=9 ttl=62 time=0.071 ms
64 bytes from 10.0.2.21: icmp_seq=10 ttl=62 time=0.076 ms

--- 10.0.2.21 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 921ms
rtt min/avg/max/mdev = 0.038/0.076/0.119/0.018 ms
root@VStreamer:/tmp/pycore.41647/VStreamer.conf#
```

Figura 2: Teste de Ping.



```
root@VStreamer:/tmp/pycore.41647/VStreamer.conf# traceroute 10.0.2.21
traceroute to 10.0.2.21 (10.0.2.21), 30 hops max, 60 byte packets
 1  10.0.0.1 (10.0.0.1)  0.435 ms  0.401 ms  0.396 ms
 2  10.0.1.2 (10.0.1.2)  0.392 ms  0.378 ms  0.372 ms
 3  10.0.2.21 (10.0.2.21)  0.357 ms  0.341 ms  0.335 ms
root@VStreamer:/tmp/pycore.41647/VStreamer.conf#
```

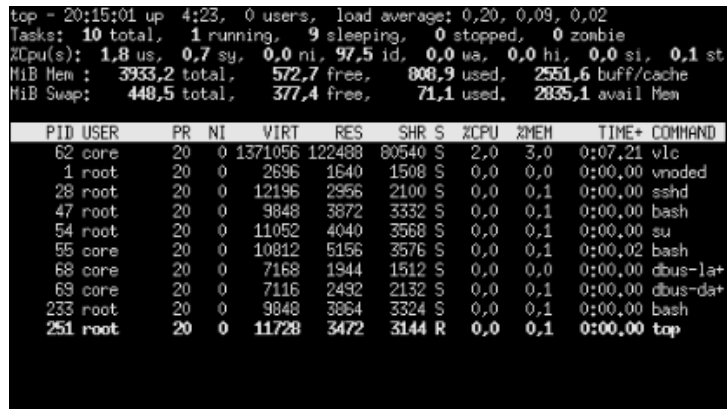
Figura 3: Teste de Traceroute.

2.2 Implementação e Análise de Tráfego

O servidor *VStreamer* foi configurado, através do *software* VLC, para realizar o *transcoding* e transmitir o ficheiro `videoA.mp4` encapsulado em Ogg/Theora via HTTP (porta 8080). Do ponto de vista teórico, embora o protocolo TCP não seja o mais eficiente para transmissões de vídeo devido aos atrasos introduzidos pelos seus mecanismos de retransmissão e controlo de congestão, o *streaming* sobre HTTP/TCP é normalmente utilizado pela sua facilidade em atravessar *firewalls* e *NATs*. Para além disso, para lidar com as variações de atraso da rede (*jitter*), o cliente de *streaming* implementa um *buffer* de reprodução. Este armazenamento temporário permite compensar o *layout*

delay, garantindo que a reprodução do vídeo se mantém fluida mesmo com ligeiras flutuações na entrega dos pacotes.

Para analisar o impacto da escalabilidade da solução, foram adicionados clientes progressivamente à rede. A monitorização do tráfego foi realizada com recurso ao Wireshark, e a carga computacional do servidor foi extraída através da ferramenta `top` (Figura 4).



```

top - 20:15:01 up 4:23, 0 users, load average: 0,20, 0,09, 0,02
tasks: 10 total, 1 running, 9 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1,8 us, 0,7 sy, 0,0 ni, 97,5 id, 0,0 wa, 0,0 hi, 0,0 si, 0,1 st
MiB Mem : 3933,2 total, 572,7 free, 808,9 used, 2551,6 buff/cache
MiB Swap: 448,5 total, 377,4 free, 71,1 used, 2835,1 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+ COMMAND
 62 core      20   0 1371056 122488 80540 S   2,0   3,0   0:07,21 vlc
   1 root        20   0    2696    1640   1508 S   0,0   0,0   0:00,00 vncd
 28 root        20   0   12196   2966   2100 S   0,0   0,1   0:00,00 sshd
 47 root        20   0    9848   3872   3332 S   0,0   0,1   0:00,00 bash
 54 root        20   0   11052   4040   3568 S   0,0   0,1   0:00,00 su
 55 core       20   0   10812   5156   3576 S   0,0   0,1   0:00,02 bash
 68 core       20   0    7168   1944   1512 S   0,0   0,0   0:00,00 dbus-la+
 69 core       20   0    7116   2432   2132 S   0,0   0,1   0:00,00 dbus-da+
233 root        20   0    9848   3864   3324 S   0,0   0,1   0:00,00 bash
251 root        20   0   11728   3472   3144 R   0,0   0,1   0:00,00 top

```

Figura 4: Carga de CPU no servidor durante a transmissão (2,0%).

Os resultados recolhidos encontram-se resumidos na Tabela 1. É importante notar que o *bitrate* de vídeo original do ficheiro `videoA.mp4` é de aproximadamente 19 kbps.

Tabela 1: Resultados dos Cenários de Streaming HTTP Unicast

Métrica	1 Cliente	2 Clientes	3 Clientes
Taxa Tráfego Total (bps)	≈ 130 kbps	≈ 262 kbps	≈ 402 kbps
Nº de Fluxos TCP	1	2	3
Utilização CPU Servidor	2,0%	2,0%	2,0%

2.3 Conclusão da Escalabilidade

Com base nas capturas efetuadas (Figuras 5, 6 e 7 em anexo), conclui-se que o modelo de *streaming* HTTP simples estabelece sessões *Unicast* independentes. Observa-se que a taxa de tráfego na interface de saída do servidor aumenta à medida que o número de clientes aumenta também (≈ 130 kbps por cada cliente adicionado).

O impacto no processamento (CPU) é baixo. No entanto, o consumo de largura de banda impede a utilização por múltiplos utilizadores, pois o

tráfego de cada nova sessão ocupa o débito disponível no *link* do servidor. Estes resultados confirmam o modelo teórico de distribuição cliente-servidor tradicional, no qual o servidor envia uma cópia independente do conteúdo para cada um dos N clientes. Se o ficheiro tiver um tamanho F e a capacidade de *upload* do servidor for u_s , o tempo total para distribuir o conteúdo aumenta linearmente segundo a expressão $\frac{N \times F}{u_s}$. Isto explica o problema de escalabilidade detetado, já que o consumo de largura de banda cresce á medida que o número de utilizadores aumenta, esgotando rapidamente a capacidade do *link* de saída do servidor.

Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B
10.0.0.1	224.0.0.5	16	1248	16	1248	0	0	0.033089	30.0100	332
10.0.0.10	10.0.0.20	780	531 k	390	505 k	390	25 k	0.000000	31.1043	130 k

Figura 5: Captura Wireshark - Cenário com 1 Cliente.

Ethernet · 4											
IPv4 · 3		IPv6 · 1		TCP · 2		UDP					
Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	
10.0.0.1	224.0.0.5	16	1248	16	1248	0	0	1.036643	30.0086		332
10.0.0.10	10.0.0.20	798	544 k	399	518 k	399	26 k	0.000000	31.5900		131 k
10.0.0.10	10.0.2.20	798	544 k	399	518 k	399	26 k	0.000059	31.5901		131 k

Figura 6: Captura Wireshark - Cenário com 2 Clientes.

Ethernet · 6		IPv4 · 4		IPv6 · 3		TCP · 3		UDP											
Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B									
10.0.0.1	224.0.0.5	8	624	8	624	0	0	0.795733	14.0049	356									
10.0.0.10	10.0.0.20	426	289 k	213	275 k	213	14 k	0.000000	16.4318	134 k									
10.0.0.10	10.0.2.20	426	289 k	213	275 k	213	14 k	0.000141	16.4317	134 k									
10.0.0.10	10.0.2.21	425	289 k	213	275 k	212	13 k	0.000212	16.4317	134 k									

Figura 7: Captura Wireshark - Cenário com 3 Clientes.

3 Etapa 2: Streaming adaptativo sobre HTTP (MPEG-DASH)

3.1 Implementação e Configuração

Nesta etapa, o objetivo foi testar o sistema de *streaming* com débito adaptativo (MPEG-DASH). O servidor VLC foi substituído por um servidor HTTP

na porta 9999, que disponibiliza os segmentos de vídeo e a descrição do conteúdo.

Com a ferramenta FFmpeg, o ficheiro `videoB.mp4` foi utilizado para criar três versões com qualidades diferentes:

- 160x100 a 200 kbps
- 320x200 a 500 kbps
- 640x400 a 1000 kbps

O ficheiro de manifesto (`video_manifest.mpd`) foi gerado com o MP4Box e contém os links para os segmentos e as regras de *bitrate*. O cliente (Firefox) utilizou a biblioteca `dash.all.debug.js` para ler o manifesto e efetuar os pedidos HTTP.

3.2 Análise do Manifesto e Funcionamento (Questões 2 e 4)

No início da reprodução, o cliente pede o ficheiro MPD e o de inicialização, como se vê na Figura 8. O ficheiro MPD funciona como um mapa da transmissão, indicando ao leitor DASH os segmentos disponíveis, a sua duração e a largura de banda necessária para cada um.

A largura de banda mínima necessária para receber a versão de menor qualidade é de 200 kbps. A pilha protocolar é composta por vídeo em MP4 sobre HTTP/TCP/IP. Ao contrário do *streaming* tradicional onde o servidor controla o envio dos dados, a arquitetura DASH centraliza a tomada de decisão no cliente. O ficheiro manifesto é o elemento central deste processo, pois fornece as opções de codificação disponíveis. É o cliente que avalia periodicamente as condições da rede e pede ativamente o próximo fragmento com a melhor qualidade que a largura de banda atual consegue suportar, utilizando para isso pedidos HTTP convencionais.

No.	Time	Source	Destination	Protocol	Length Info
6	3.300496674	10.0.2.20	10.0.0.10	HTTP	501 GET /video_dash.html HTTP/1.1
8	3.300932756	10.0.0.10	10.0.2.20	HTTP	272 HTTP/1.1 304 Not Modified
14	3.346922184	10.0.2.20	10.0.0.10	HTTP	439 GET /dash.all.debug.js HTTP/1.1
16	3.347474039	10.0.0.10	10.0.2.20	HTTP	272 HTTP/1.1 304 Not Modified
22	3.454791650	10.0.2.20	10.0.0.10	HTTP	502 GET /video_manifest.mpd HTTP/1.1
24	3.455131577	10.0.0.10	10.0.2.20	HTTP	273 HTTP/1.1 304 Not Modified
30	3.571395291	10.0.2.20	10.0.0.10	HTTP	419 GET /video_manifest_init.mp4 HTTP/1.1
32	3.571804007	10.0.0.10	10.0.2.20	HTTP	257 HTTP/1.1 304 Not Modified

Figura 8: Captura Wireshark: Pedido inicial do ficheiro manifesto e bibliotecas JavaScript.

3.3 Adaptação ao Congestionamento de Rede (Questão 3)

Para testar o mecanismo DASH, criou-se um cenário com limitações na rede. O cliente *Aladdin* manteve uma ligação sem limites, enquanto a largura de banda do cliente *Bela* foi limitada a 300 kbps (Figuras 9 e 10).

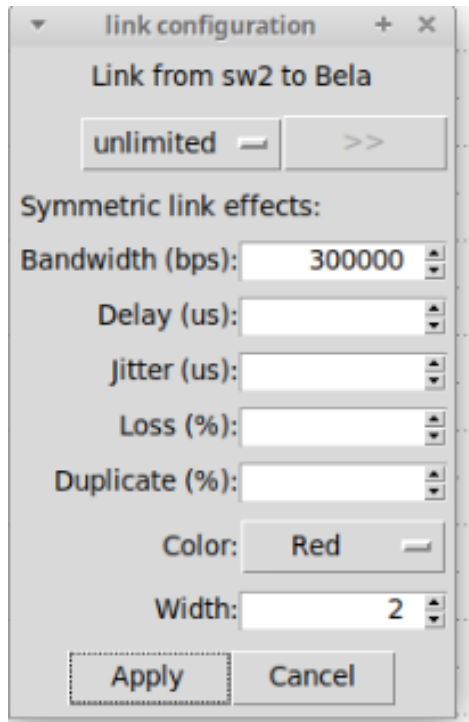


Figura 9: Configuração do limite de largura de banda para 300 kbps.

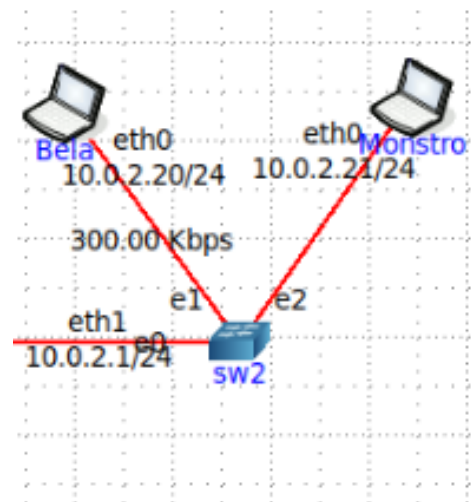


Figura 10: Topologia CORE com indicação do limite aplicado.

As capturas de tráfego mostram o comportamento do sistema. No início, com largura de banda disponível, o nó *Bela* pede os fragmentos de qualidade máxima (1000k), como se vê na Figura 11.

5716	38.605343771	10.0.2.20	10.0.0.10	HTTP	401 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
7277	38.612761434	10.0.0.10	10.0.2.20	MP4	173
7284	38.667443535	10.0.2.20	10.0.0.10	HTTP	402 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
8579	38.680608202	10.0.0.10	10.0.2.20	MP4	173
8586	38.734583459	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
9885	38.748322322	10.0.0.10	10.0.2.20	MP4	173
9892	38.794311587	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
11546	38.801319518	10.0.0.10	10.0.2.20	MP4	173
11553	38.845716546	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
13038	38.862439336	10.0.0.10	10.0.2.20	MP4	173
13045	38.912018172	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
14247	38.926480547	10.0.0.10	10.0.2.20	MP4	173
14254	38.966977935	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
15804	38.976372712	10.0.0.10	10.0.2.20	MP4	173
15811	39.021559528	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
17340	39.035291518	10.0.0.10	10.0.2.20	MP4	173
17347	39.082450947	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
18653	39.095559870	10.0.0.10	10.0.2.20	MP4	173
18660	39.1610221671	10.0.2.20	10.0.0.10	HTTP	403 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
20065	39.170066455	10.0.0.10	10.0.2.20	MP4	173

Figura 11: Tráfego no nó Bela antes da limitação: pedidos dos segmentos de 1000 kbps.

Depois de aplicar o limite de 300 kbps, o cliente DASH deteta o aumento do *delay* e altera os pedidos HTTP. A Figura 12 mostra a adaptação: o cliente deixa de pedir a versão de 1000 kbps, passa pela de 500 kbps e, como o limite se manteve, acaba por pedir a de 200 kbps (*videoB_160_100_200k*).

38	3.603692951	10.0.2.20	10.0.0.10	HTTP	399 GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
119	5.063308715	10.0.2.20	10.0.0.10	HTTP	398 GET /videoB_320_200_500k_dash.mp4 HTTP/1.1
801	17.597373060	10.0.0.10	10.0.2.20	MP4	573
918	24.630546695	10.0.2.20	10.0.0.10	HTTP	399 GET /videoB_160_100_200k_dash.mp4 HTTP/1.1
1222	30.372539705	10.0.0.10	10.0.2.20	MP4	891
1259	32.566453089	10.0.2.20	10.0.0.10	HTTP	400 GET /videoB_160_100_200k_dash.mp4 HTTP/1.1
1562	38.267597942	10.0.0.10	10.0.2.20	MP4	891
1600	40.498210319	10.0.2.20	10.0.0.10	HTTP	400 GET /videoB_160_100_200k_dash.mp4 HTTP/1.1
1795	44.420463438	10.0.2.20	10.0.0.10	HTTP	402 GET /videoB_160_100_200k_dash.mp4 HTTP/1.1
1861	45.162891244	10.0.2.20	10.0.0.10	HTTP	402 GET /videoB_160_100_200k_dash.mp4 HTTP/1.1
2192	51.828941984	10.0.0.10	10.0.2.20	MP4	891

Figura 12: Tráfego no nó Bela após a limitação: alteração progressiva para a versão de 200 kbps.

O nó *Aladdin*, sem limites na rede, continuou a pedir continuamente os fragmentos da versão de maior qualidade (Figura 13).

No.	Time	Source	Destination	Protocol	Length	Info
47	65.849910924	10.0.0.21	10.0.0.10	HTTP	423	GET /video_dash.html HTTP/1.1
49	65.850108114	10.0.0.10	10.0.0.21	HTTP	562	HTTP/1.1 200 Ok (text/html)
55	65.893306432	10.0.0.21	10.0.0.10	HTTP	389	GET /dash.all.debug.js HTTP/1.1
3658	65.932756993	10.0.0.10	10.0.0.21	HTTP	796	HTTP/1.1 200 Ok (application/x-javascript)
3665	66.157768191	10.0.0.21	10.0.0.10	HTTP	440	GET /favicon.ico HTTP/1.1
3667	66.158068373	10.0.0.10	10.0.0.21	HTTP	741	HTTP/1.1 404 Not Found (text/html)
3674	66.164948869	10.0.0.21	10.0.0.10	HTTP	470	GET /video_manifest.mpd HTTP/1.1
3690	66.165320881	10.0.0.10	10.0.0.21	HTTP/X...	209	HTTP/1.1 200 Ok
3696	66.283019464	10.0.0.21	10.0.0.10	HTTP	371	GET /video_manifest_init.mp4 HTTP/1.1
3698	66.283442269	10.0.0.10	10.0.0.21	MP4	1060	
3705	66.296329286	10.0.0.21	10.0.0.10	HTTP	411	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
4950	66.310367877	10.0.0.10	10.0.0.21	MP4	173	
4957	66.416141890	10.0.0.21	10.0.0.10	HTTP	413	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
6507	66.424555360	10.0.0.10	10.0.0.21	MP4	173	
6514	66.466598438	10.0.0.21	10.0.0.10	HTTP	413	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
7690	66.474697815	10.0.0.10	10.0.0.21	MP4	173	
7697	66.501261105	10.0.0.21	10.0.0.10	HTTP	414	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
9484	66.525980987	10.0.0.10	10.0.0.21	HTTP	415	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
10746	66.529870818	10.0.0.21	10.0.0.10	MP4	173	
10753	66.553772720	10.0.0.10	10.0.0.21	HTTP	415	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
11931	66.558827232	10.0.0.21	10.0.0.10	MP4	173	
11938	66.586279524	10.0.0.10	10.0.0.21	HTTP	415	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1
13545	66.592119872	10.0.0.21	10.0.0.10	MP4	173	
13551	66.614935942	10.0.0.10	10.0.0.21	HTTP	415	GET /videoB_640_400_1000k_dash.mp4 HTTP/1.1

Figura 13: Tráfego no nó Aladdin: pedidos da versão de 1000 kbps.

Análise de QoE (*Quality of Experience*)

Na análise da experiência do utilizador (*QoE*), verificam-se resultados diferentes devido ao DASH. No nó *Aladdin* (Figura 14), a largura de banda permitiu a resolução de 640x400, resultando na melhor qualidade de imagem. No nó *Bela* (Figura 15), a diminuição da largura de banda levou à escolha da versão de 160x100.

Embora a imagem tenha pior qualidade, o mecanismo DASH ajustou o débito para evitar falhas no *buffer* e manter o vídeo disponível para o cliente. Este ajustamento demonstra o princípio teórico base do *streaming* adaptativo: dar prioridade à continuidade da reprodução em detrimento da resolução da imagem. Ao baixar a taxa de bits assim que deteta um aumento no atraso da rede, o cliente garante que os novos fragmentos são transferidos a tempo de serem reproduzidos. Esta estratégia impede a falta de dados no *buffer* (fenómeno conhecido por *starvation*), impedindo paragens abruptas e melhorando significativamente a experiência do utilizador perante instabilidades na rede.

Streaming ESR: etapa 2 DASH

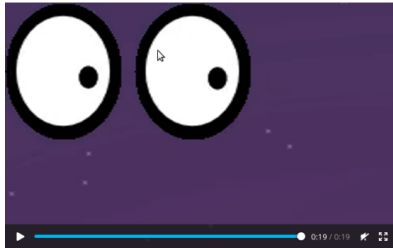


Figura 14: QoE no Aladdin: Qualidade 640x400.

Streaming ESR: etapa 2 DASH

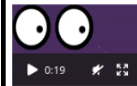


Figura 15: QoE no Bela: Qualidade 160x100.

4 Etapa 3: Streaming RTP/RTCP em Unicast e Multicast

4.1 Configuração e Análise Unicast

Nesta etapa, analisámos o envio de vídeo sobre o protocolo UDP. O servidor iniciou a transmissão *Unicast* para o cliente *Monstro* (IP 10.0.2.21, porta 5555), gerando localmente o ficheiro de descrição da sessão (SDP) para configuração do cliente.

Para verificar o tráfego no Wireshark, configurámos a ferramenta para interpretar os datagramas UDP como RTP. Este passo permitiu observar o encapsulamento correto do vídeo, conforme documentado nas Figuras 16 e 17. A utilização do RTP sobre o UDP justifica-se pela falta de mecanismos de ordenação e garantia de entrega no protocolo de transporte tradicional. Do ponto de vista teórico, o cabeçalho RTP adiciona elementos que são essenciais para o *streaming* em tempo real: o *Timestamp*, que representa a base temporal das amostras e permite ao cliente sincronizar o áudio/vídeo e calcular o *jitter*, e o *Sequence Number*, utilizado para detetar pacotes perdidos ou entregues fora de ordem. Para além disso, o protocolo RTCP atua em conjunto com o protocolo RTP para enviar mensagens de controlo (*Sender* e *Receiver Reports*) que fornecem um *feedback* contínuo sobre a qualidade da rede.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.10	10.0.2.21	UDP	1514	44954 → 5555 Len=1472
2	0.000082475	10.0.0.10	10.0.2.21	UDP	144	44954 → 5555 Len=102
3	0.050966065	10.0.0.10	10.0.2.21	UDP	858	44954 → 5555 Len=816
4	0.101907016	10.0.0.10	10.0.2.21	UDP	862	44954 → 5555 Len=820
5	0.152896642	10.0.0.10	10.0.2.21	UDP	503	44954 → 5555 Len=461
6	0.203878281	10.0.0.10	10.0.2.21	UDP	1046	44954 → 5555 Len=1004
7	0.254969530	10.0.0.10	10.0.2.21	UDP	712	44954 → 5555 Len=670
8	0.306001055	10.0.0.10	10.0.2.21	UDP	670	44954 → 5555 Len=628
9	0.357082978	10.0.0.10	10.0.2.21	UDP	552	44954 → 5555 Len=510
10	0.398293586	10.0.0.10	10.0.2.21	UDP	1514	44954 → 5555 Len=1472

Figura 16: Pacotes UDP enviados para o cliente (Unicast).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.10	10.0.2.21	RTP	1514	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7727, Time=293735...
2	0.000082475	10.0.0.10	10.0.2.21	RTP	144	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7728, Time=293735...
3	0.050966065	10.0.0.10	10.0.2.21	RTP	858	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7729, Time=293735...
4	0.101907016	10.0.0.10	10.0.2.21	RTP	862	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7730, Time=293735...
5	0.152896642	10.0.0.10	10.0.2.21	RTP	503	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7731, Time=293736...
6	0.203878281	10.0.0.10	10.0.2.21	RTP	1046	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7732, Time=293736...
7	0.254969530	10.0.0.10	10.0.2.21	RTP	712	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7733, Time=293737...
8	0.306001055	10.0.0.10	10.0.2.21	RTP	670	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7734, Time=293737...
9	0.357082978	10.0.0.10	10.0.2.21	RTP	552	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7735, Time=293738...
10	0.398293586	10.0.0.10	10.0.2.21	RTP	1514	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7736, Time=293738...
11	0.398368574	10.0.0.10	10.0.2.21	RTP	1514	PT=DynamicRTP-Type-96, SSRC=0x32F2EC13, Seq=7737, Time=293738...

Figura 17: Tráfego interpretado como RTP.

Ao nível do tráfego, as estatísticas confirmam que a transmissão para um único cliente consome aproximadamente 209 kbps (Figura 18). A entrada de um segundo cliente na sessão *Unicast* (IP 10.0.2.20) resultou na duplicação do tráfego gerado na interface do servidor, atingindo cerca de 424 kbps (Figura 19).

Ethernet - 3		IPv4 - 2		IPv6 - 1		TCP		UDP - 2					
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	
10.0.0.10	44954	10.0.2.21	5555	793	756 k	793	756 k	0	0	0.000000	28.8074	209 k	
10.0.0.10	44955	10.0.2.21	5556	6	420	6	420	0	0	2.999113	25.0572	134	

Figura 18: Estatísticas UDP: Tráfego gerado para 1 cliente Unicast (≈ 209 kbps).

Ethernet - 3		IPv4 - 3		IPv6 - 1		TCP		UDP - 4					
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	
10.0.0.10	48721	10.0.2.20	5555	797	767 k	797	767 k	0	0	0.000000	29.0036	211 k	
10.0.0.10	51094	10.0.2.21	5555	799	771 k	799	771 k	0	0	0.018954	28.9509	213 k	

Figura 19: Estatísticas UDP: Tráfego gerado para 2 clientes Unicast simultâneos (≈ 424 kbps).

4.2 Configuração e Análise Multicast

Para testar a distribuição *Multicast*, simplificámos a topologia para um único *switch* a interligar o servidor e os quatro clientes (Figura 20).

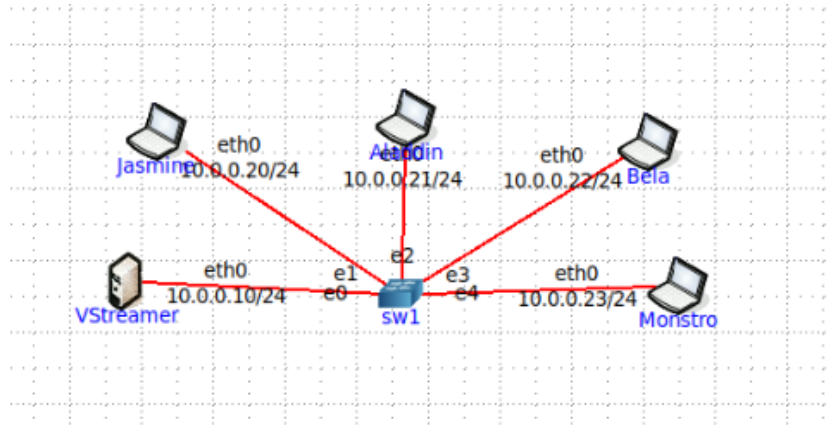


Figura 20: Topologia de rede utilizada no cenário Multicast.

O servidor transmitiu o fluxo de vídeo para o endereço de grupo 224.0.0.200. Diferente do modo *Unicast*, a descoberta da sessão pelos clientes fez-se através do protocolo SAP (*Session Announcement Protocol*). A Figura 21 comprova o envio do anúncio SDP para o endereço 224.2.127.254, e também onde se verifica que o fluxo de vídeo é apresentado como MP4V-ES na coluna de protocolo, isto porque o Wireshark processa o anúncio SAP/SDP inicial e identifica o tipo de dados contido nos pacotes RTP seguintes. Por outro lado, a Figura 22 detalha o fluxo de dados UDP enviado para o endereço *Multicast*.

53	1.896106125	10.0.0.10	224.2.127.254	SAP/SDP	366 Announcement (v1)
54	1.896207162	10.0.0.10	224.0.0.200	RTP	78 Sender Report
55	1.896235819	10.0.0.10	224.0.0.200	MP4V-ES	639 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4831, Time=2837786598, Mark
56	1.947618471	10.0.0.10	224.0.0.200	MP4V-ES	646 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4832, Time=2837791098, Mark
57	1.998757033	10.0.0.10	224.0.0.200	MP4V-ES	712 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4833, Time=2837795598, Mark
58	2.050161268	10.0.0.10	224.0.0.200	MP4V-ES	783 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4834, Time=2837800098, Mark
59	2.091387220	10.0.0.10	224.0.0.200	MP4V-ES	868 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4835, Time=2837804598, Mark
60	2.142918035	10.0.0.10	224.0.0.200	MP4V-ES	1514 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4836, Time=2837809098
61	2.143007275	10.0.0.10	224.0.0.200	MP4V-ES	1514 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4837, Time=2837809098
62	2.143018789	10.0.0.10	224.0.0.200	MP4V-ES	1514 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4838, Time=2837809098
63	2.143029959	10.0.0.10	224.0.0.200	MP4V-ES	1514 PT=MP4V-ES, SSRC=0x6871ED37, Seq=4839, Time=2837809098

Figura 21: Pacote de anúncio SAP/SDP e transmissão da *stream* de vídeo (MP4V-ES).

5 Etapa 4: Redes de Overlay e Distribuição de Conteúdo

5.1 Implementação da Rede Base (Underlay)

A infraestrutura física simulada no CORE (*Underlay*) consiste num núcleo com cinco *routers* interligados, configurados com o protocolo de encaminhamento dinâmico OSPF. A topologia inclui quatro redes locais distintas (LAN A a LAN D). O servidor (*ServerStreamer*) encontra-se na LAN A, e o cliente final (*PC8*) localiza-se na LAN D, conforme ilustrado na Figura 24.

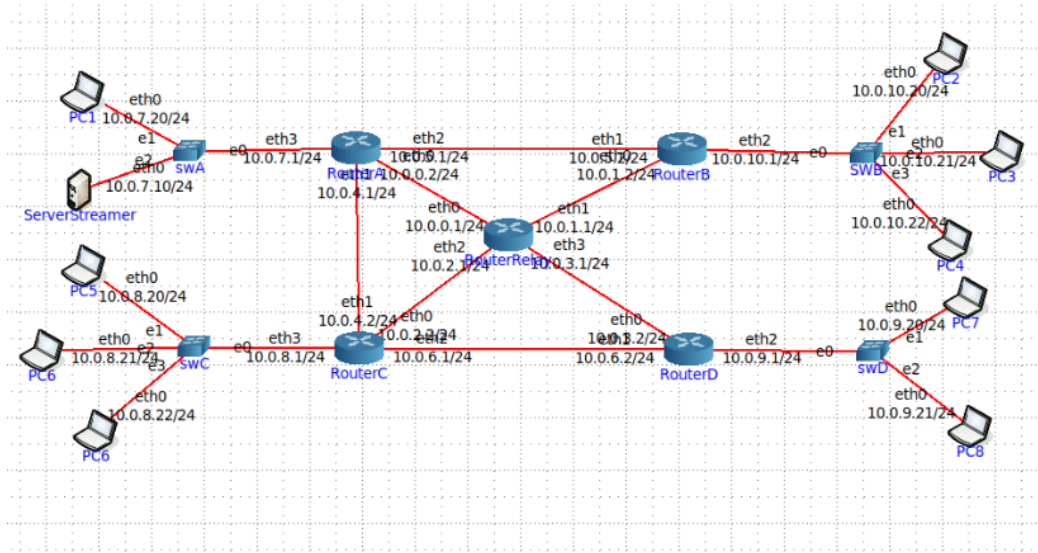


Figura 24: Topologia base da rede com 5 *routers* e múltiplos nós.

O encaminhamento IP foi validado através de testes `ping` e `traceroute` entre o servidor e o cliente final. A Figura 26 comprova a conectividade e demonstra os quatro saltos (*hops*) efetuados pelos pacotes no núcleo da rede.


```
vcmd
root@ServerStreamer:/tmp/pycore.36693/ServerStreamer.conf# ping -c 10 10.0.9.21
PING 10.0.9.21 (10.0.9.21) 56(84) bytes of data.
64 bytes from 10.0.9.21: icmp_seq=1 ttl=61 time=0.144 ms
64 bytes from 10.0.9.21: icmp_seq=2 ttl=61 time=0.137 ms
64 bytes from 10.0.9.21: icmp_seq=3 ttl=61 time=0.129 ms
64 bytes from 10.0.9.21: icmp_seq=4 ttl=61 time=0.132 ms
64 bytes from 10.0.9.21: icmp_seq=5 ttl=61 time=0.205 ms
64 bytes from 10.0.9.21: icmp_seq=6 ttl=61 time=0.108 ms
64 bytes from 10.0.9.21: icmp_seq=7 ttl=61 time=0.165 ms
64 bytes from 10.0.9.21: icmp_seq=8 ttl=61 time=0.145 ms
64 bytes from 10.0.9.21: icmp_seq=9 ttl=61 time=0.121 ms
64 bytes from 10.0.9.21: icmp_seq=10 ttl=61 time=0.133 ms

--- 10.0.9.21 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 919ms
rtt min/avg/max/mdev = 0.108/0.141/0.205/0.025 ms
root@ServerStreamer:/tmp/pycore.36693/ServerStreamer.conf#
```

Figura 25: Teste ping: Servidor para PC8.

```
vcmd
<core.36693/ServerStreamer.conf# traceroute 10.0.9.21
traceroute to 10.0.9.21 (10.0.9.21), 30 hops max, 60 byte packets
 1 10.0.7.1 (10.0.7.1) 0.414 ms 0.348 ms 0.339 ms
 2 10.0.0.1 (10.0.0.1) 0.330 ms 0.295 ms 0.273 ms
 3 10.0.3.2 (10.0.3.2) 0.257 ms 0.222 ms 0.205 ms
 4 10.0.9.21 (10.0.9.21) 0.190 ms 0.156 ms 0.132 ms
root@ServerStreamer:/tmp/pycore.36693/ServerStreamer.conf#
```

Figura 26: Teste traceroute: Caminho pela rede.

5.2 Implementação da Rede Overlay

Sobre a infraestrutura IP, configurou-se uma rede *Overlay*. Selecionou-se o nó *RouterRelay* para atuar como nó retransmissor.

A transmissão foi feita utilizando o formato MPEG-TS diretamente sobre UDP, sem usar o protocolo RTP. No servidor, iniciou-se o envio de vídeo para o endereço IP do *RouterRelay*:

```
ffmpeg -re -i videoB.mp4 -f mpegts -c copy udp://10.0.0.1:5000
```

No nó retransmissor, o FFmpeg recebeu os dados na porta 5000 e reen-caminhou o vídeo para o cliente final:

```
ffmpeg -i udp://@:5000 -f mpegts -c copy udp://10.0.9.21:5555
```

Este formato divide o fluxo de vídeo em pacotes de tamanho fixo com 188 bytes. A presença de timestamps (como PCR, PTS e DTS) garante a sincronização e a reconstrução do vídeo no cliente, o que diminui as variações e os atrasos provocados ao longo dos múltiplos saltos na rede.

5.3 Análise de Desempenho

O uso do *router* para retransmitir o vídeo consome recursos locais no equipamento. A monitorização do sistema (Figura 27) indica que o processo `ffmpeg` ocupou 0,7% da CPU.

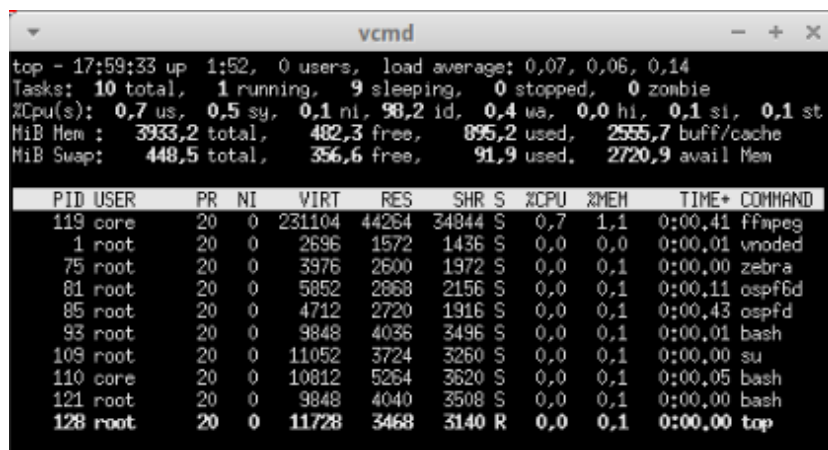


Figura 27: Consumo de CPU no nó retransmissor provocado pelo processo FFmpeg.

Para avaliar o atraso de transmissão inserido pelo *router*, procedeu-se à identificação do mesmo fragmento de vídeo nas interfaces de entrada e saída através do cabeçalho MPEG-TS/PES.

No.	Time	Source	Destination	Protocol	Length	Info
1	18:02:03,385742	fe80::200:ff:feaa:e	ff02::5	OSPF	98	Hello Packet
2	18:02:03,513643	10.0.7.1	224.0.0.5	OSPF	78	Hello Packet
3	18:02:05,514064	10.0.7.1	224.0.0.5	OSPF	78	Hello Packet
4	18:02:07,514693	10.0.7.1	224.0.0.5	OSPF	78	Hello Packet
5	18:02:09,515075	10.0.7.1	224.0.0.5	OSPF	78	Hello Packet
6	18:02:11,515815	10.0.7.1	224.0.0.5	OSPF	78	Hello Packet
7	18:02:13,221755	10.0.7.10	10.0.0.1	UDP	1514	39531 → 5000 Len=1472
8	18:02:13,221867	10.0.7.10	10.0.0.1	UDP	1514	39531 → 5000 Len=1472
9	18:02:13,221881	10.0.7.10	10.0.0.1	UDP	1046	39531 → 5000 Len=1004
10	18:02:13,221914	10.0.7.10	10.0.0.1	MPEG TS	230	39531 → 5000 Len=188 [MP2T fragment of a reassembled packet]
11	18:02:13,221930	10.0.7.10	10.0.0.1	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
12	18:02:13,221946	10.0.7.10	10.0.0.1	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
13	18:02:13,272646	10.0.7.10	10.0.0.1	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
14	18:02:13,323558	10.0.7.10	10.0.0.1	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
15	18:02:13,375433	10.0.7.10	10.0.0.1	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]

Figura 28: Wireshark: Entrada do fragmento de vídeo no nó retransmissor (13.221914s).

No.	Time	Source	Destination	Protocol	Length	Info
1	18:02:07,514350	10.0.9.1	224.0.0.5	OSPF	78	Hello Packet
2	18:02:09,514718	10.0.9.1	224.0.0.5	OSPF	78	Hello Packet
3	18:02:11,515061	10.0.9.1	224.0.0.5	OSPF	78	Hello Packet
4	18:02:13,221969	10.0.3.1	10.0.9.21	MPEG TS	230	54538 → 5555 Len=188 [MP2T fragment of a reassembled packet]
5	18:02:13,222190	10.0.3.1	10.0.9.21	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
6	18:02:13,222191	10.0.3.1	10.0.9.21	UDP	1514	54538 → 5555 Len=1472
7	18:02:13,222192	10.0.3.1	10.0.9.21	UDP	1514	54538 → 5555 Len=1472
8	18:02:13,222193	10.0.3.1	10.0.9.21	RTCP	858	Payload-specific Feedback TSTR [Malformed Packet]
9	18:02:13,222194	10.0.3.1	10.0.9.21	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
10	18:02:13,272939	10.0.3.1	10.0.9.21	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
11	18:02:13,323853	10.0.3.1	10.0.9.21	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
12	18:02:13,375820	10.0.3.1	10.0.9.21	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
13	18:02:13,426972	10.0.3.1	10.0.9.21	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]
14	18:02:13,478572	10.0.3.1	10.0.9.21	MPEG PES	230	video-stream [MP2T fragment of a reassembled packet]

Figura 29: Wireshark: Saída do fragmento reencaminhado para o cliente (13.221969s).

As capturas mostram que o pacote entrou no nó no instante $t_{in} = 13.221914$ e foi reencaminhado no instante $t_{out} = 13.221969$. Este processo gerou um atraso local de $55\mu s$.

5.4 Conclusões Comparativas

A arquitetura *Overlay* oferece flexibilidade, uma vez que funciona sem depender da existência de *Multicast* na rede física. No entanto, esta solução transfere o processamento para a CPU do equipamento, o que introduz um atraso de transmissão de dezenas de microssegundos em cada nó. Este comportamento resulta em perdas de desempenho face à distribuição *Multicast* feita pelo *hardware* de rede. Apesar deste atraso de processamento, as redes *Overlay* funcionam como uma alternativa ao encaminhamento da infraestrutura IP subjacente (*Underlay*). A criação destas redes na camada de aplicação permite definir rotas específicas e serve de base para o funcionamento das *Content Delivery Networks* (CDNs). Desta forma, torna-se possível distribuir servidores com cópias do conteúdo para locais mais próximos dos utilizadores, o que diminui o tempo de acesso e evita a sobrecarga de um servidor central.

6 Conclusões

Este trabalho permitiu observar as características técnicas de várias arquiteturas de distribuição de vídeo.

O *streaming* sobre HTTP simples (Etapa 1) mostrou-se ineficiente para múltiplos utilizadores, dado que o tráfego gerado aumenta com o número de ligações. O formato MPEG-DASH (Etapa 2) contornou as limitações de largura de banda nos *links* de acesso, uma vez que o cliente ajusta a qualidade do vídeo aos atrasos da rede, o que garante a visualização do vídeo sem interrupções.

Na avaliação de protocolos sobre UDP (Etapa 3), comprovou-se que o *Multicast* diminui a carga na rede física ao enviar um fluxo único que o *switch* encaminha apenas para os clientes registados no grupo. Perante a necessidade de configuração dos equipamentos de rede para o *Multicast*, a Etapa 4 demonstrou que as redes *Overlay* são uma alternativa na camada de aplicação. A limitação desta abordagem é o tempo de processamento nos nós intermédios, que adiciona um atraso de transmissão.

A principal dificuldade técnica foi a compatibilidade do navegador Firefox com o manifesto DASH, uma vez que o navegador deixou de aceitar o ficheiro após uma atualização. Notou-se também uma divergência na descrição da Etapa 4, que mencionava o uso do protocolo RTP quando os comandos aplicavam o formato MPEG-TS sobre UDP. Como trabalho futuro, sugere-se a introdução de perda de pacotes nos *links* virtuais para avaliar o comportamento do protocolo UDP face ao TCP na transmissão de vídeo.

7 Referências

Referências

- [1] CORE Emulator. *Common Open Research Emulator*. Disponível em: <https://github.com/coreemu/core>
- [2] Wireshark. *Network Protocol Analyzer*. Disponível em: <https://www.wireshark.org/>
- [3] FFmpeg. *Multimedia Framework*. Disponível em: <https://ffmpeg.org/>
- [4] VideoLAN. *VLC Media Player*. Disponível em: <https://www.videolan.org/>
- [5] DASH Industry Forum. *Dash.js Player*. Disponível em: <https://dashif.org/>

8 Declaração de Uso de IA e Responsabilidade

Ferramentas de IA usadas: Google Gemini (maio de 2026).

Link da conversa: <https://gemini.google.com/share/a3b9067c7426>

Prompts e respostas relevantes: Foram apresentadas três questões sobre a análise de fluxos de vídeo no Wireshark:

- **Prompt 1:** Dúvida sobre o motivo de o tráfego RTP aparecer apenas como UDP na Etapa 3 (*Unicast*). **Resposta:** Foi explicado que a porta 5555 não está associada por defeito ao RTP e indicou-se o uso da função *Decode As* para atualizar a visualização do protocolo.
- **Prompt 2:** Questão sobre o aparecimento do nome MP4V-ES nos pacotes de *Multicast*. **Resposta:** Explicou-se que o Wireshark lê o anúncio SAP/SDP inicial e identifica o fluxo com base nessa descrição.
- **Prompt 3:** Pedido de clarificação sobre as diferenças entre MPEG-ES, MPEG-PES e MPEG-TS na Etapa 4 face ao MP4V-ES da Etapa 3. **Resposta:** Foram detalhadas as fases de processamento do vídeo e justificou-se a necessidade do formato MPEG-TS sobre UDP para manter a sincronização.

Aplicação no trabalho: A inteligência artificial foi utilizada para esclarecer dúvidas sobre os formatos de transmissão e a apresentação de dados no analisador de pacotes. A montagem da rede no emulador CORE, a introdução de comandos e a recolha de dados foram feitas de forma manual.

Declaração: Assume-se a total responsabilidade pelos conteúdos e pelo código apresentados neste relatório, confirmando-se a compreensão do funcionamento dos protocolos e das opções implementadas.